

Event-Oriented Application with RxJava

Student Worksheet

Introduction

In this worksheet, we will apply the concepts and techniques learned about RxJava to generate results by processing event streams. To do this, we will use the `EventStreams` class, which declares 4 types of events and 4 methods that simulate the generation of these event types with different characteristics. In the `App` class, we have the `main()` method, which provides a menu to test the following methods.

1 Exercise Statements

Exercise 1. Method `clean()`

Implement the `clean()` method to clean the `TrafficSensorEvent` observable so that only events where the number of vehicles is non-negative and the average speed is greater than 0 are considered valid. Additionally, there should be no events with duplicate `timestamps`.

Exercise 2. Method `weatherErrorRecovery()`

Implement the `weatherErrorRecovery()` method to manage potential errors occurring in the `WeatherEvent` observable. The management consists of retrying 3 times to obtain the stream without errors; if this fails, emit the event `WeatherEvent("DEFAULT", 20.0, Instant.now())`.

Important Note

Each of the following methods must return the `Disposable` object obtained from the subscription.

Exercise 3. Method `speedPerRoad()`

Implement the `speedPerRoad()` method which, using a `TrafficSensorEvent` observable, should calculate and display the average speed for each road, sampled every 5 seconds.

Expected output:

```
C1 -> 8.778831998611647
A2 -> 53.807720490879234
B1 -> 11.024349582805574
...
```

Exercise 4. Method `avgSpeedLastSeconds()`

Implement the `avgSpeedLastSeconds()` method which, using a `TrafficSensorEvent` observable, should obtain and display the maximum average speed for each 5-second window.

Expected output:

```
Window max speed=58.44
Window max speed=67.32
Window max speed=81.54
```

Exercise 5. Method `combineIncidentsWeather()`

Implement the `combineIncidentsWeather()` method, which receives observables of type `IncidentEvent` and `WeatherEvent` to combine them so that for each incident event, the weather at that instant is added.

Exercise 6. Method `maxDistanceBatch()`

Implement the `maxDistanceBatch()` method, which processes a `VehiclePosition` flowable by setting a backpressure strategy to store new events that the subscriber cannot process. Using the `distance()` method, which calculates the distance between two coordinates in kilometers, you must obtain the maximum distance in each 500ms window. The processing part of each window's events must be performed on the `computation()` scheduler.

Exercise 7. Method `highTraffic()`

Implement the `highTraffic()` method, which takes a `TrafficSensorEvent` observable as an argument to display high traffic events, defined as those where the number of vehicles is greater than 60. To avoid false alarms, events followed by another event within 2 seconds must be discarded.

Exercise 8. Method `congestionAlert()`

Implement the `congestionAlert()` method, which receives a `TrafficSensorEvent` observable and displays a congestion alert per road. A congestion is detected when the average `avgSpeed` value in sets of 10 events per road type is less than 40.